

Funny Prank Writeup

26nd August 2025

Prepared By: Hybread

Challenge Author: Hybread

Difficulty: Hard

Writeup classification: Official

1 – Introduction

Funny Prank is a hard cryptography challenge which implements a custom defined emoji written decryptor. This decryptor is used to decrypt an encrypted .png file which is also given to the user with a .EMOJI tail format. The decryptor features a 2 layer Mersenne Twister PRNG algorithm and affine cipher.

2 – Description

My friend came to my house the other day, and bro thought he was a funny guy by encrypting my important picture where I use to store my MOST important passwords. I didn't know about this until today which when I opened my directory, I was greeted with 3 files in it. I'm getting a little sick and tired of his pranks...

3 – Skills Required

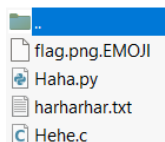
- Basic knowledge of obfuscation tactics
- Able to read python code
- Good knowledge of python scripting
- Good knowledge of ciphers
- Understanding of Mersenne Twister and PRNG recovery

4 – Skills Learnt

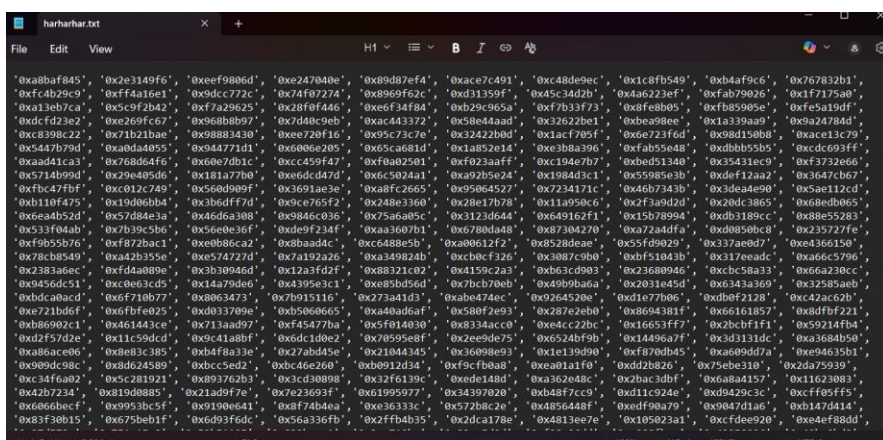
- Learn about goofy obfuscation techniques such as this challenge
- Learn about Mersenne Twister algorithm
- Brute forcing affine cipher

5 – Solve

Download the zip file and be greeted with 4 different files:



Flag.png seems to be a corrupted .png file with a .EMOJI header, hinting that it's been encrypted. Opening 'harharhar.txt' we see that it's a bunch of hex values with no further explanation:



For the other files, in 'Hehe.c' we get a bunch of defines:



While in 'Haha.py' we see a bunch of emojis in a standard code indented format

```
1  ?
2
3  (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
4  (((A * x + B) % 256) x G)
5
6  (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
7  = (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
8  = [G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
9  = (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
10 = (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
11 = (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
12 = (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
13 = (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
14 = (G: (A: (B: (C: (D: (E: (F: (G: (H: (I: (J: (K: (L: (M: (N: (O: (P: (Q: (R: (S: (T: (U: (V: (W: (X: (Y: (Z: (
15
16 = b"\x89PNG\r\n\x1a\n"
17
18 == " ":
19 ("")
20
```

To begin, we'll map out each emoji, or ask AI to do it (it does a pretty good job). Then we get the actual code of an encryption script:

```
1  from random import Random
2
3  def veryMath(buf: bytes, a: int, b: int) -> bytes:
4      return bytes(((a * x + b) % 256) for x in buf)
5
6  def veryMoreMath(buf: bytes, seed: int) -> tuple[list[int], bytes]:
7      r = Random(seed)
8      leak = [r.getrandbits(32) for _ in range(624)]
9      ks = bytearray()
10     while len(ks) < len(buf):
11         ks.extend(r.getrandbits(32).to_bytes(4, 'big'))
12     ks = bytes(ks[:len(buf)])
13     ct = bytes((u ^ v) for u, v in zip(buf, ks))
14     return leak, ct
15
16     PNG_MAGIC = b"\x89PNG\r\n\x1a\n"
17
18     if __name__ == "__main__":
19         print("")
```

We can see that there's 2 functions, presumably a 2-layer encryption on the image. First the 'veryMath' function, by looking at it, we can deduce that function's instructions to be of an Affine Cipher. For the 'veryMoreMath' function, we see that the script uses python's Random class and calling 'r.getrandbits(32)' to generate a specific 32-bit output, which then generates 624 outputs in hex with the following 'for _ in range(624)', where 624 is a big hint indicator of the MT19937 algorithm. These output are exactly what the 'harharhar.txt' contains. The intermediate buffer is then XOR'ed with a keystream generated by the MT19937 algorithm with using an unknown seed.

By google searching, we can find that in any scenario, a MT19937 leak is always tempered (bit-scrambled). Further searching reveals that MT19937 uses a specific tempering function, and to inverse it, we can use python's Random class to implement a mini untempering function. But to do that, we'll need to clone the given MT19937 function, set the internal state using the untempered values, and apply the inverse temper which we'll then be able generate the actual keystream. We'll create a new solve script, and implement this function as such:

```
# Untemper function from Cryptopals Challenge 23
def untemper(y):
    def int_to_bit_list(x):
        return [int(b) for b in '{:032b}'.format(x)]

    def bit_list_to_int(l):
        return int(''.join(str(x) for x in l), base=2)

    def invert_shift_mask_xor(y, direction, shift, mask=0xFFFFFFFF):
        y_bits = int_to_bit_list(y)
        mask_bits = int_to_bit_list(mask)

        if direction == 'left':
            y_bits.reverse()
            mask_bits.reverse()

        x = [None] * 32
        for n in range(32):
            if n < shift:
                x[n] = y_bits[n]
            else:
                x[n] = y_bits[n] ^ (mask_bits[n] & x[n - shift])

        if direction == 'left':
            x.reverse()

        return bit_list_to_int(x)

    u, d = 11, 0xFFFFFFFF
    s, b = 7, 0x9D2C5680
    t, c = 15, 0xEFC60000
    l = 18

    xx = y
    xx = invert_shift_mask_xor(xx, direction='right', shift=1)
    xx = invert_shift_mask_xor(xx, direction='left', shift=t, mask=c)
    xx = invert_shift_mask_xor(xx, direction='left', shift=s, mask=b)
    xx = invert_shift_mask_xor(xx, direction='right', shift=u, mask=d)

    return xx
```

```
def main():
    with open("harharhar.txt", "r") as f:
        text = f.read().strip()
    leak_hex = ast.literal_eval(text.split("LEAK_HEX=")[1])
    leak = [int(h, 16) for h in leak_hex]

    # Untempering
    state = [untemper(x) for x in leak]

    # Clone the Random object and position after the leak
    r = random.Random()
    full_state = tuple(state) + (624,)
    r.setstate((3, full_state, None))

    with open("flag.png.EMOJI", "rb") as f:
        ct = f.read()

    # Generate keystream
    ks = bytearray()
    while len(ks) < len(ct):
        ks.extend(r.getrandbits(32).to_bytes(4, 'big'))
    ks = bytes(ks[:len(ct)])
```

Next, we have our 2nd Affine layer. Since we're only given the equation, we'll need to implement an affine cipher brute force function. The affine transformation in 'veryMath' is $y = (a * x + b) \% 256$. To invert it, we compute $x = ((y - b) * \text{inv_a}) \% 256$, where 'inv_a' is the modular inverse of a modulo 256. Since 'a' is odd (the coprime to 256), try all odd a (1 to 255) and all b (0 to 255):

```
# Modular inverse using extended Euclidean algorithm
def egcd(a, m):
    if a == 0:
        return m, 0, 1
    g, y, x = egcd(m % a, a)
    return g, x - (m // a) * y, y

def modinv(a, m):
    g, x, y = egcd(a, m)
    if g != 1:
        raise Exception('No modular inverse')
    return x % m
```

```
found = False
for a in range(1, 256, 2): # Odd a (coprime to 256)
    try:
        inv_a = modinv(a, 256)
    except Exception:
        continue # Skip if no inverse (though odd a should have)
    for b in range(256):
        # Invert affine: x = (c - b) * inv_a mod 256
        raw = bytes(((x - b) % 256 * inv_a) % 256 for x in c1)
        if raw.startswith(PNG_MAGIC):
            with open("recovered_flag.png", "wb") as f:
                f.write(raw)
            print(f"Recovered PNG with a={a}, b={b}. Open 'recovered_flag.png' to view the flag.")
            found = True
            break
    if found:
        break
if not found:
    print("No valid affine parameters found.")
```

After that, we run the script and sure enough we get the recovered image:

```
PS C:\Users\Hybread\Documents\Uni stuff\CTF\ents\Uni stuff\CTF\CYNX\Lecture's Over, T
Found keystream: 219f799681c3aa9a...
Found affine values: a=5, b=123
Recovered PNG. Open 'recovered_flag.png'.
```

recovered_flag.png

CYNX{cRyp70_3m0ji_h3l1}

Flag = CYNX{cRyp70_3m0ji_h3l1}